

Introducción

Los patrones de diseño son soluciones generales y reutilizables para problemas que aparecen con frecuencia durante el desarrollo de software. No representan código que deba copiarse exactamente, sino formas de organizar clases, objetos y responsabilidades para conseguir aplicaciones más ordenadas, mantenibles y flexibles.

Los patrones de diseño clásicos suelen dividirse en tres categorías principales: creacionales, relacionados con la creación de objetos; estructurales, enfocados en la forma en que se relacionan clases y objetos; y de comportamiento, que se concentran en la comunicación y distribución de responsabilidades entre los objetos.

Patrones Creacionales

Criterio	Singleton	Factory Method	Abstract Factory	Builder
Categoría	Creacional	Creacional	Creacional	Creacional
Problema que resuelve	Evita que existan varias instancias de una clase cuando solamente se necesita una.	Evita que el código dependa directamente de una clase concreta al crear objetos.	Permite crear familias de objetos relacionados sin especificar sus clases concretas.	Facilita la creación de objetos que requieren muchos datos o diferentes configuraciones.
Objetivo principal	Garantizar una única instancia y proporcionar un punto de acceso a ella. Microsoft describe Singleton como un patrón que restringe una clase a una instancia.	Delegar la creación de objetos para no tener que indicar directamente la clase exacta que será creada.	Proporcionar una interfaz para crear grupos de objetos relacionados sin depender de sus implementaciones concretas.	Construir un objeto paso a paso, separando su construcción de su representación final.
Funcionamiento general	La clase controla su propio objeto. Normalmente utiliza un constructor privado y un método que devuelve siempre	Se define un método encargado de crear objetos. Las clases derivadas pueden decidir qué implementación concreta crear.	Se crea una fábrica general con métodos para producir diferentes objetos relacionados. Cada fábrica concreta crea una familia específica.	Se utiliza un objeto Builder al cual se le van indicando diferentes características y al final se ejecuta un método como build(). La API oficial de Java utiliza este enfoque, por

	la misma instancia.			ejemplo, con <code>HttpRequest.Builder</code> .
Ventajas	Evita objetos innecesarios; permite compartir recursos; proporciona acceso centralizado.	Reduce el acoplamiento; facilita agregar nuevas clases; hace más flexible la creación de objetos.	Garantiza compatibilidad entre objetos de una misma familia; facilita cambiar familias completas de objetos.	Facilita la creación de objetos complejos; hace el código más legible; permite diferentes configuraciones del mismo objeto.
Desventajas	Puede generar dependencia global; dificulta algunas pruebas; un uso excesivo puede generar fuerte acoplamiento.	Agrega más clases e interfaces; puede resultar innecesario para aplicaciones pequeñas.	Puede generar muchas clases y aumentar considerablemente la complejidad del proyecto.	Requiere crear clases u objetos adicionales; puede ser excesivo para objetos simples.
Cuándo utilizarlo	Cuando solamente debe existir una instancia compartida de determinado servicio o recurso.	Cuando una aplicación debe crear diferentes tipos de objetos pero no debe depender directamente de sus clases.	Cuando existen varias familias de componentes que deben trabajar entre sí.	Cuando un objeto tiene muchos parámetros opcionales o su creación requiere varios pasos.
Caso de uso	Configuración global, administrador de conexión o sistema de registro de eventos.	Sistema de notificaciones que pueda crear correo, SMS o notificaciones móviles.	Aplicación gráfica con componentes diferentes para Windows, macOS o Linux.	Construcción de usuarios, solicitudes HTTP, productos o configuraciones complejas.
Ejemplo en proyecto real	Una aplicación puede utilizar una sola instancia de	Una tienda puede tener <code>NotificacionFactory</code> para crear Email,	Un sistema multiplataforma puede tener <code>WindowsFactory</code> y	Java utiliza <code>HttpRequest.Builder</code> para configurar URI, encabezados, cuerpo y

	ConexionBD para administrar el acceso a la base de datos.	SMS o Push dependiendo de la preferencia del usuario.	MacFactory, capaces de producir botones, ventanas y menús del mismo estilo.	otros datos antes de construir una solicitud HTTP.
Complejidad	Baja	Media	Alta	Media

Comparación entre los patrones creacionales

La principal diferencia es qué parte de la creación de objetos intenta controlar cada patrón. Singleton controla la cantidad de instancias; Factory Method decide qué clase concreta crear; Abstract Factory administra familias completas de objetos relacionados; y Builder se enfoca en construir objetos complejos paso a paso.

Factory Method y Abstract Factory están relacionados, pero Abstract Factory trabaja normalmente con varios tipos de objetos que forman una familia, mientras que Factory Method suele concentrarse en la creación de un tipo de producto. Microsoft Learn distingue precisamente Factory Method como una forma de crear objetos sin indicar la clase exacta y Abstract Factory como una forma de encapsular grupos de fábricas relacionadas.

Patrones Estructurales

Criterio	Adapter	Facade	Decorator
Categoría	Estructural	Estructural	Estructural
Problema que resuelve	Permite trabajar juntas a clases que tienen interfaces incompatibles.	Reduce la dificultad de utilizar un sistema formado por muchas clases o subsistemas.	Permite agregar nuevas responsabilidades a un objeto sin modificar directamente su clase.
Objetivo principal	Convertir la interfaz de una clase en otra interfaz que la aplicación pueda utilizar. Microsoft lo describe como una forma de utilizar la interfaz de una clase existente como otra interfaz.	Proporcionar una interfaz sencilla para acceder a un sistema más complejo.	Agregar comportamiento a objetos individuales sin modificar el comportamiento de todos los objetos de la misma clase.
Funcionamiento general	Se crea una clase intermedia que recibe las peticiones del sistema y las transforma al formato esperado por otro componente.	Una clase Facade recibe solicitudes sencillas y se encarga de comunicarse con diferentes clases internas.	El objeto original es envuelto por otro objeto que mantiene su interfaz pero agrega una función antes o después de ejecutar la operación original.

Ventajas	Permite reutilizar código antiguo; facilita integrar APIs o librerías externas; reduce modificaciones en código existente.	Simplifica el código; reduce dependencias; facilita el uso de sistemas complejos.	Agrega funcionalidades dinámicamente; evita crear demasiadas subclases; permite combinar diferentes comportamientos.
Desventajas	Introduce clases adicionales; muchos adaptadores pueden dificultar la arquitectura.	Una Facade demasiado grande puede terminar concentrando demasiadas responsabilidades.	Muchos decoradores encadenados pueden dificultar la lectura y depuración del código.
Cuándo utilizarlo	Al integrar sistemas, APIs o librerías que utilizan estructuras diferentes.	Cuando un subsistema tiene muchas operaciones y se quiere proporcionar una entrada más sencilla.	Cuando es necesario agregar funcionalidades sin modificar la clase original.
Caso de uso	Conectar una aplicación nueva con un servicio de pagos antiguo.	Proceso de compra que involucra inventario, pagos, envíos y notificaciones.	Agregar compresión, cifrado, registro o validaciones a diferentes operaciones.
Ejemplo en proyecto real	Una tienda utiliza una interfaz Pago, pero PayPal tiene una API diferente. PayPalAdapter convierte las llamadas de la tienda al formato de PayPal.	Una clase CompraFacade puede ejecutar verificarInventario(), procesarPago(), crearEnvio() y enviarCorreo() mediante una sola operación realizarCompra().	Un sistema de notificaciones puede comenzar con una notificación básica y agregar decoradores para enviar además SMS o registrar la notificación en un archivo.
Complejidad	Media	Baja-Media	Media

Comparación entre los patrones estructurales

Aunque Adapter y Facade pueden parecer similares porque ambos colocan una clase entre el cliente y otro sistema, su propósito es diferente. Adapter cambia una interfaz para hacerla compatible, mientras que Facade simplifica una interfaz que ya existe. Microsoft Learn establece esta diferencia al presentar Adapter como un mecanismo de adaptación de interfaces y Facade como una interfaz simplificada hacia un conjunto mayor de código.

Decorator tiene otro propósito: no busca adaptar ni simplificar, sino agregar responsabilidades adicionales a un objeto sin cambiar su clase original.

Patrones de Comportamiento

Criterio	Observer	Strategy	Command
Categoría	Comportamiento	Comportamiento	Comportamiento
Problema que resuelve	Mantener sincronizados varios objetos cuando cambia el estado de otro objeto.	Evitar tener un algoritmo fijo cuando existen diferentes maneras de realizar una misma operación.	Evitar que quien solicita una acción tenga que conocer directamente cómo se realiza.
Objetivo principal	Crear una relación donde varios objetos sean informados automáticamente cuando otro cambia. Microsoft describe al sujeto como el objeto que mantiene una lista de observadores y los notifica cuando cambia su estado.	Encapsular diferentes algoritmos y poder seleccionar uno de ellos durante la ejecución.	Convertir una acción o solicitud en un objeto que contiene la información necesaria para ejecutarla.
Funcionamiento general	Un objeto llamado sujeto mantiene una colección de observadores. Cuando ocurre un cambio, el sujeto envía una notificación a los observadores registrados.	Cada algoritmo implementa una interfaz común. El objeto principal recibe una estrategia y utiliza su implementación sin conocer los detalles internos.	Cada operación se representa mediante un objeto Command. Otro objeto puede ejecutar, almacenar o administrar esos comandos.
Ventajas	Reduce dependencias directas; permite notificaciones automáticas; facilita agregar nuevos observadores.	Permite cambiar algoritmos fácilmente; evita grandes estructuras if o switch; mejora la reutilización.	Desacopla al emisor del receptor; permite guardar acciones; facilita implementar colas, historial, deshacer y rehacer.
Desventajas	Muchos observadores pueden dificultar saber qué componentes recibirán una actualización; pueden generarse notificaciones innecesarias.	Aumenta el número de clases; para algoritmos muy simples puede ser innecesario.	Cada acción puede requerir una nueva clase; aumenta el número de objetos del sistema.

Cuándo utilizarlo	Cuando varios componentes dependen de cambios realizados en otro objeto.	Cuando existen varias soluciones para realizar una misma tarea y se quiere poder intercambiarlas.	Cuando las operaciones deben almacenarse, ejecutarse posteriormente, ponerse en cola o deshacerse.
Caso de uso	Sistemas de notificaciones, interfaces gráficas, suscripciones y actualización de información.	Métodos de pago, cálculo de descuentos, algoritmos de rutas o diferentes formas de ordenar información.	Botones de aplicaciones, sistemas de tareas, editores con deshacer/rehacer o procesamiento de trabajos pendientes.
Ejemplo en proyecto real	En una tienda, cuando cambia el estado de un pedido, el sistema puede notificar al cliente, actualizar la pantalla y registrar el cambio.	Una tienda puede tener estrategias PagoTarjeta, PagoPayPal y PagoTransferencia. El usuario selecciona una y el sistema cambia el algoritmo de pago.	En un editor de texto, acciones como CopiarCommand, PegarCommand y EliminarCommand pueden almacenarse para posteriormente implementar la opción de deshacer.
Complejidad	Media	Media	Media-Alta

Comparación entre los patrones de comportamiento

Los tres patrones permiten reducir el acoplamiento, pero solucionan problemas diferentes.

Observer se concentra en la comunicación entre objetos. Cuando cambia un objeto, los demás reciben una notificación automáticamente.

Strategy se concentra en los algoritmos. Permite cambiar la forma de realizar una operación durante la ejecución sin modificar el objeto que utiliza el algoritmo.

Command se concentra en las acciones. Una petición se representa mediante un objeto independiente que contiene la información necesaria para ejecutarla, lo que facilita características como historial de acciones o ejecución posterior.

Comparación general

Categoría	¿En qué se concentra?	Pregunta que ayuda a identificarla	Patrones analizados
Creacionales	Creación de objetos.	¿Cómo se debe crear este objeto?	Singleton, Factory Method, Abstract Factory y Builder
Estructurales	Organización y relación entre objetos y clases.	¿Cómo puedo conectar u organizar estos componentes?	Adapter, Facade y Decorator
Comportamiento	Comunicación y responsabilidades entre objetos.	¿Cómo deben interactuar estos objetos?	Observer, Strategy y Command

Una manera sencilla de recordar las tres categorías es:

Creacional = crear.

Estructural = organizar.

Comportamiento = comunicar o actuar.

Conclusión

Los patrones de diseño permiten utilizar soluciones que ya han demostrado ser útiles ante problemas comunes en el desarrollo de software. Sin embargo, no existe un patrón que sea mejor para todas las situaciones. La elección depende principalmente del problema que se desea resolver.

Los patrones creacionales permiten controlar y flexibilizar la creación de objetos. Los patrones estructurales ayudan a organizar y conectar diferentes componentes de una aplicación. Finalmente, los patrones de comportamiento permiten definir de una forma más organizada cómo se comunican los objetos y cómo se realizan diferentes acciones.

También es importante considerar que utilizar un patrón de diseño aumenta, en algunos casos, la cantidad de clases e interfaces del proyecto. Por este motivo, los patrones deben utilizarse cuando realmente ayudan a solucionar un problema y no solamente para hacer que un programa parezca más complejo.

Comprender las diferencias entre Singleton, Factory Method, Abstract Factory, Builder, Adapter, Facade, Decorator, Observer, Strategy y Command permite seleccionar soluciones adecuadas dependiendo de las necesidades de cada aplicación y facilita el desarrollo de sistemas más mantenibles y fáciles de modificar.

Bibliografía

- Singleton — Microsoft Learn:
<https://learn.microsoft.com/en-us/shows/visual-studio-toolbox/design-patterns-singleton>
- Factory Method y Abstract Factory — Microsoft Learn:
<https://learn.microsoft.com/en-us/shows/visual-studio-toolbox/design-patterns-factories>
- Builder — Documentación oficial de Java (Oracle):
<https://docs.oracle.com/en/java/javase/26/docs/api/java.net.http/java/net/http/HttpRequest.Builder.html>
- Adapter y Facade — Microsoft Learn:
<https://learn.microsoft.com/en-us/shows/visual-studio-toolbox/design-patterns-adapterfaade>
- Decorator — Microsoft Learn:
<https://learn.microsoft.com/en-us/shows/visual-studio-toolbox/design-patterns-decorator>
- Observer — Microsoft Learn:
<https://learn.microsoft.com/en-us/shows/visual-studio-toolbox/design-patterns-observer-publish-subscribe>
- Strategy — Microsoft Learn:
<https://learn.microsoft.com/en-us/shows/visual-studio-toolbox/design-patterns-strategy>
- Command — Microsoft Learn:
<https://learn.microsoft.com/en-us/shows/visual-studio-toolbox/design-patterns-commandmemento>